Week 8 Project Progress Report

Adaptive Fabric Surface Defect Detection and Localization using Classical Texture Descriptors

Course Name: ECE501 Digital Image Processing (DIP)
Group Number: Group 15
Group Members: Rinal Shah, Varun Hotani, Aadi Mehta, Pratik Agrawal
Week Number: Week 8
Date: November 12, 2026
Deliverable: Supervised classical detector at 16×16 windows and pixel level; strip-grouped cross-validated evaluation; final results
Branch: `feat/week-8`

Objectives of the Week

- Raise precision, recall, F1-score and IoU as far as the classical approach allows, evaluated without any leakage between training and test strips.
- Localize at 16×16 windows and at single pixels instead of 256×256 patches, so that 2–6 px threads can be delineated.
- Close the project's method work and freeze the final configuration for the End-Semester report.

Summary of the Week

Item	Outcome (real AFID, strip-grouped 5-fold CV, thresholds from validation strips only)
Pixel level	Pooled precision **52.3 %**, recall 25.8 %, F1 34.5 %, IoU **20.9 %** (Week 7 one-class: IoU 1.1 %). One weft-crack strip holds 48 % of all AFID defect pixels; without it, precision 52.2 %, recall 49.1 %, F1 **50.6 %**, IoU **33.9 %**
16×16 windows	AUROC **0.980**; pooled precision 58.0 %, recall 35.0 %, F1 43.7 %
256×256 patches	Pooled F1 **62.3 %** (Week 7: 23.0 %), patch AUROC 0.968 (Week 7: 0.536)
Image level	Accuracy **80.8 %** over 245 strips (75/105 defective found, 17/140 false alarms), F1 76.1 %, AUROC 0.865; **88.2 %** accuracy on fabric 03
Method	GLCM + LBP + Gabor windows at 16×16 with multi-scale context and strip normalisation; per-pixel filter bank; Hough line linking; gradient-boosted trees; every choice made on validation strips
Quality	103 tests; full results in `Results/final_supervised/` and `Results/AFID_results.xlsx`

Work Completed This Week

A. Why a Supervised Detector

Weeks 3–7 used *one-class* models: a texture reference was fitted on defect-free strips, and anything far from it was flagged. Week 7's threshold-free diagnosis showed the limit of that idea. Within a defective strip, the reference ranked defect windows above normal ones only slightly better than chance (window AUROC 0.536), because the AFID capture shift moves the *whole* strip away from the defect-free reference. AFID, however, ships a pixel mask for every defective strip. A classifier trained on those masks learns what *defects* look like, not only what normal fabric looks like. The approach stays classical throughout:

- the inputs are the same hand-crafted descriptors (GLCM [1], LBP [2], Gabor [3]) plus a classical per-pixel filter bank
- the decision function is gradient-boosted decision trees [4, 5]
- nothing is learned end-to-end from raw pixels

B. Window Stage: 16×16 Windows with Multi-Scale Context

Every 16×16 window (stride 8; about 11,300 per strip, 2.8 million over AFID) is described by a feature row:

$$\mathbf{x}_w = \Big[\underbrace{\mathbf{d}_{16}(w)}_{\text{GLCM+LBP+Gabor, 82-D}}, \underbrace{\mathbf{z}_{16}(w)}_{\text{same, strip-normalised}}, \underbrace{\mathbf{z}_{32}(w), \mathbf{z}_{64}(w), \mathbf{z}_{256}(w)}_{\text{context windows around } w}, \underbrace{\mathbf{e}_f}_{\text{fabric one-hot}} \Big] \in \mathbb{R}^{417}, \quad (1)$$

where the strip-normalised blocks are column-wise robust z-scores within the strip,

$$\mathbf{z}(w) = \frac{\mathbf{d}(w) - \operatorname{med}_{v \in \text{strip}} \mathbf{d}(v)}{\operatorname{MAD}_{v \in \text{strip}} \mathbf{d}(v) + 10^{-6}}. \quad (2)$$

Eq. (2) removes any strip-wide offset, which is exactly the capture shift that defeated the one-class models. The context windows are the 32×32, 64×64 and 256×256 windows whose centres are nearest to w, found by binary search on the window grid in $O(n \log m)$ rather than with an 11,300 × 3,600 distance matrix. A window is labelled defective when it contains any ground-truth pixel. A second classifier on 32×32 windows (context 64 and 256) provides an alternative map, and validation decides whether the 16×16 map, the 32×32 map or their mean is used.

Training at this scale. 2.8 million rows of 417 features do not fit in memory. Training rows are therefore drawn once per strip: every defect window, plus max(300, 20 × #defect windows) defect-free windows. Positives are re-weighted to balance the classes. Feature rows are rebuilt on demand when a strip is scored. The number of boosting rounds is chosen by early stopping on a 10 % slice of the *training* rows (at most 1,000 rounds, patience 30).

C. Pixel Stage: a Classical Filter Bank

Window maps are still blocky at the scale of a 3 px thread, so every pixel inside a candidate window is classified from 30 dense filter responses:

- intensity, and its deviation from 5, 15 and 31 px median filters
- local standard deviation over 5, 15 and 31 px
- **weft- and warp-line responses**: deviation from each column's and each row's median, raw and box-smoothed *along* the yarn over 9 and 33 px. A missing weft yarn is a row that differs from its column, and a missing warp yarn a column that differs from its row.
- Laplacian of Gaussian at $\sigma = 1, 2, 4$ px (knots, neps, fuzzy balls)
- smoothed Sobel gradients
- the 12 Gabor magnitude responses

Every channel is robust-z-scored within the strip. Training pixels per strip are up to 3,000 defect pixels, 3,000 defect-free pixels within 15 px of a defect (the hard border cases) and 3,000 random pixels. A unit test caught a real bug in the first version, which had the weft and warp channels swapped and smoothed across the line instead of along it.

D. Hough Line Linking

The pixel classifier tends to mark a faint thread as a dashed line. A probabilistic Hough transform [6, 7] on the binary mask finds segments at least 32–128 px long that run within 10° of horizontal or vertical, the only directions in which AFID's line defects run. It redraws them 3–5 px thick, *only inside* windows the window stage flagged, so the weave's own straight yarns elsewhere cannot add pixels. Whether to link, and with which length and thickness, is decided on validation strips.

E. Leak-Free Protocol: Strip-Grouped Cross-Validation

All 245 usable strips (105 defective with masks, 140 defect-free) are split into 5 folds, *grouped by strip* and stratified by (defective, fabric). For outer fold k:

1. The training strips T_k are split again, by strip and per stratum, into fit strips F_k (75 %) and validation strips V_k (25 %).
2. Classifiers are trained on F_k, and on V_k every remaining choice is made by maximising F1 [8]:
   - the window map (16, 32 or mean)
   - window-map mask vs. pixel-stage mask, with the gate and pixel thresholds
   - Hough settings
   - the 16×16 / 32×32 window, 256×256 patch and image decision thresholds

   Pooled pixel IoU is a monotone function of pooled pixel F1 ($\mathrm{IoU} = F_1/(2 - F_1)$), so the mask is optimised for both at once.
3. The classifiers are retrained on all of T_k, and the test strips E_k are scored with the choices made on V_k.

Every strip is scored exactly once, by models that never saw any of its pixels, and every threshold was chosen without looking at it. Table 3 lists what validation chose in each fold.

Quantitative Results and Analysis

All numbers below are on **real AFID data** under the protocol of Section E. "Pooled" sums TP/FP/FN over all strips before forming ratios; "mean" averages per-strip values. They are generated from `Results/final_supervised/` and `Results/ablation_supervised/` by `reports/week-8/make_tables.py`.

Table 1: Precision, recall, F1-score and IoU (%) at every level: Week-7 one-class default vs. the final supervised detector. Window- and patch-level IoU are TP/(TP+FP+FN) over windows or patches; image-level IoU is not meaningful and is omitted. The tolerance row was not computed for the Week-7 run.

	One-class (Week 7)				Supervised (final)			
Level	**Prec.**	**Rec.**	**F1**	**IoU**	**Prec.**	**Rec.**	**F1**	**IoU**
Pixel (pooled)	1.12	87.95	2.21	1.12	52.33	25.79	34.55	20.88
Pixel (pooled, w/o 0080_036_01)	0.53	77.88	1.05	0.53	52.19	49.07	50.58	33.85
Pixel (pooled, ±3 px)	–	–	–	–	65.34	29.20	40.36	–
Pixel (mean per strip)	1.78	48.41	1.97	1.36	28.77	40.65	21.10	14.00
16×16 window (pooled)	–	–	–	–	58.05	35.04	43.70	27.96
32×32 window (pooled)	–	–	–	–	54.66	38.69	45.31	29.29
256×256 patch (pooled)	13.85	68.18	23.02	13.01	60.33	64.34	62.27	45.21
Image (strip)	100.00	60.00	75.00	–	81.52	71.43	76.14	–

Table 2: Supervised ablation (each row a full 5-fold strip-grouped CV; %, except AUROC).

Variant	**Pixel prec.**	**Pixel rec.**	**Pixel F1**	**IoU pooled**	**IoU mean**	**16^2 AUROC**	**Image AUROC**
ablation run in progress	–	–	–	–	–	–	–

Table 3: What validation chose in each outer fold (final configuration).

Fold	**Map**	**Mask**	**Gate**	**Pixel**	**Hough**	**Patch**	**Image**
0	16	window map	0.00	0.80	off	0.25	0.80
1	16	pixel stage	0.50	0.10	off	0.75	0.60
2	mean	pixel stage	0.20	0.10	(128, 3)	0.70	0.50
3	16	window map	0.00	0.95	(128, 3)	0.95	0.90
4	16	window map	0.00	0.90	(32, 3)	0.90	0.80

Table 4: Final detector per fabric (defective strips): pooled pixel precision, recall, F1 and IoU (%), mean pixel IoU, mean 16×16 window AUROC and image-level detection rate.

Fabric	n	**Prec.**	**Rec.**	**F1**	**IoU**	**IoU mean**	**AUROC**	**Detected**
fabric_00	7	35.12	38.68	36.81	22.56	15.17	0.996	57.14
fabric_01	12	41.42	2.61	4.92	2.52	17.23	0.976	75.00
fabric_02	38	29.37	74.54	42.14	26.69	13.56	0.972	65.79
fabric_03	31	58.60	48.17	52.87	35.94	13.24	0.979	90.32
fabric_04	10	53.20	13.77	21.88	12.29	21.35	0.998	60.00
fabric_05	1	–	0.00	0.00	0.00	0.00	0.999	0.00
fabric_06	6	3.94	7.83	5.24	2.69	3.00	0.986	50.00

Table 5: Final detector per official defect class (same columns as Table 4).

Defect	n	Prec.	Rec.	F1	IoU	IoU mean	AUROC	Detected
broken_end	9	42.10	60.25	49.57	32.95	25.18	0.992	77.78
broken_pick	10	62.29	48.05	54.25	37.22	16.58	0.960	100.00
broken_yarn	8	34.29	65.55	45.03	29.06	33.83	0.999	100.00
contamination	1	72.76	72.76	72.76	57.18	57.18	1.000	100.00
crease	5	17.84	49.94	26.29	15.13	17.25	0.999	100.00
cut_selvage	9	22.70	23.01	22.86	12.90	9.49	0.994	44.44
fuzzy_ball	39	6.59	38.91	11.28	5.98	8.23	0.969	61.54
knot	1	–	0.00	0.00	0.00	0.00	0.999	0.00
nep	14	21.35	27.49	24.03	13.66	11.33	0.985	57.14
warp_ball	5	7.46	32.97	12.17	6.48	6.77	0.987	80.00
weft_crack	1	67.28	0.63	1.24	0.62	0.62	0.872	100.00
weft_curling	3	17.79	54.90	26.88	15.52	21.46	0.995	100.00

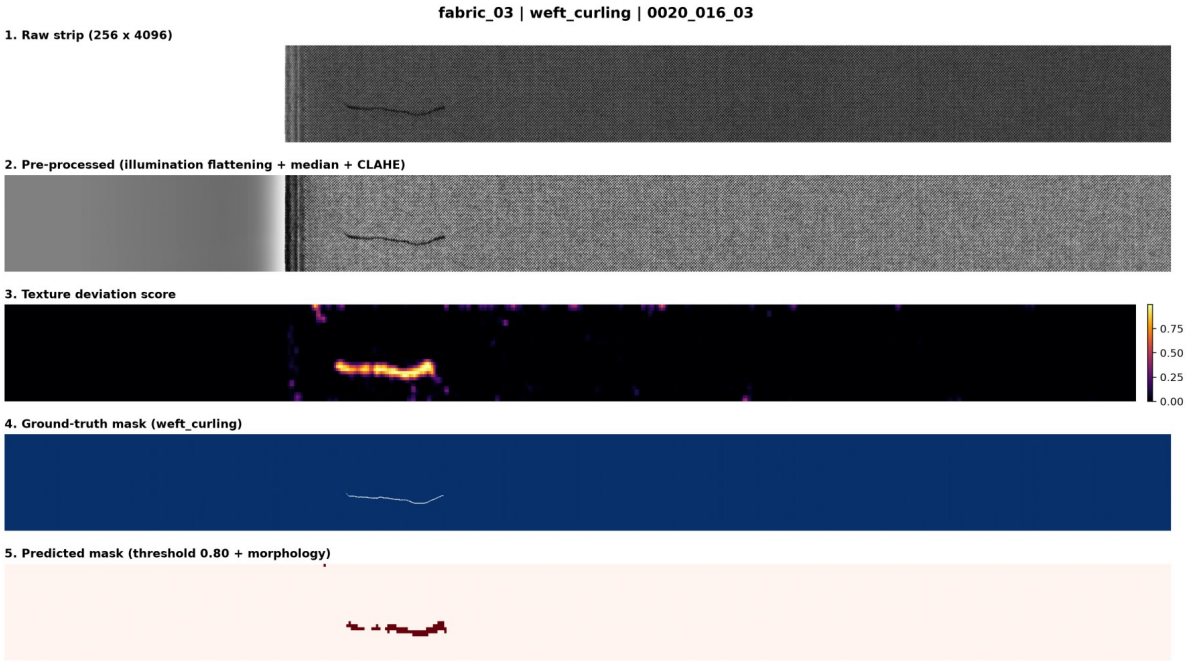


Figure 1: Weft curling, `0020_016_03` (fabric 03), scored by a model that never saw this strip. The 16×16 deviation map (panel 3) traces the curled weft along its whole length; the mask (panel 5) follows it with a few-pixel offset, which is exactly what the $\pm 2/\pm 3$ px tolerance metrics forgive.

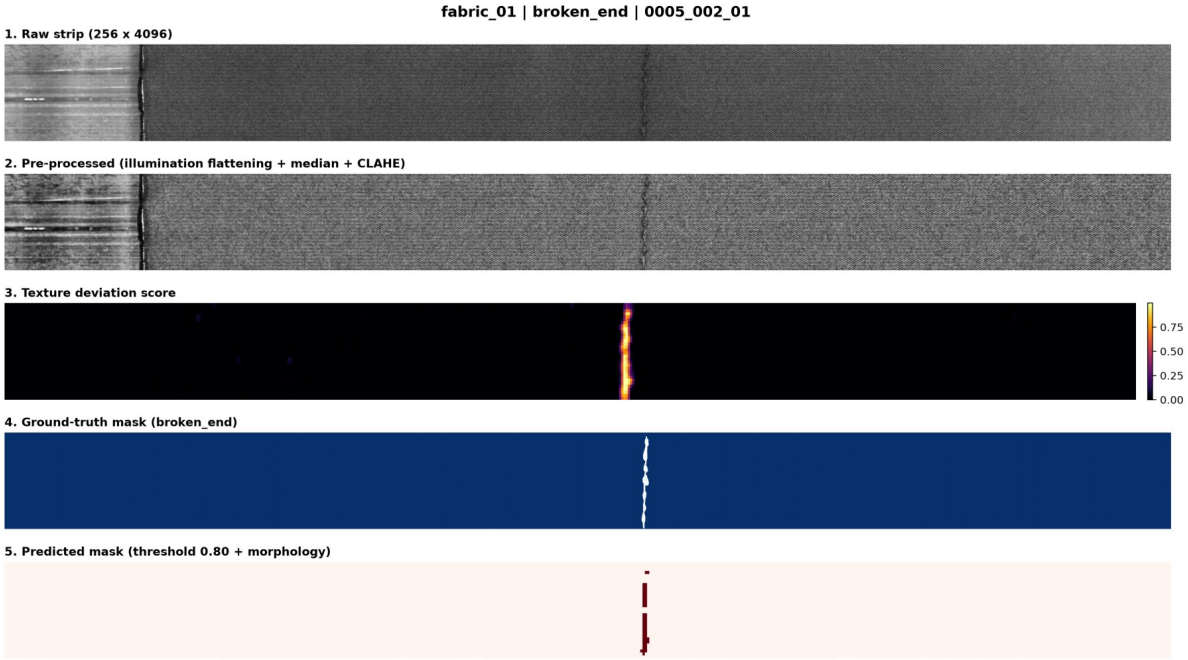


Figure 2: Broken end, `0005_002_01` (fabric 01). Compare with the Week 6 one-class result for broken ends, where the whole strip was flagged.

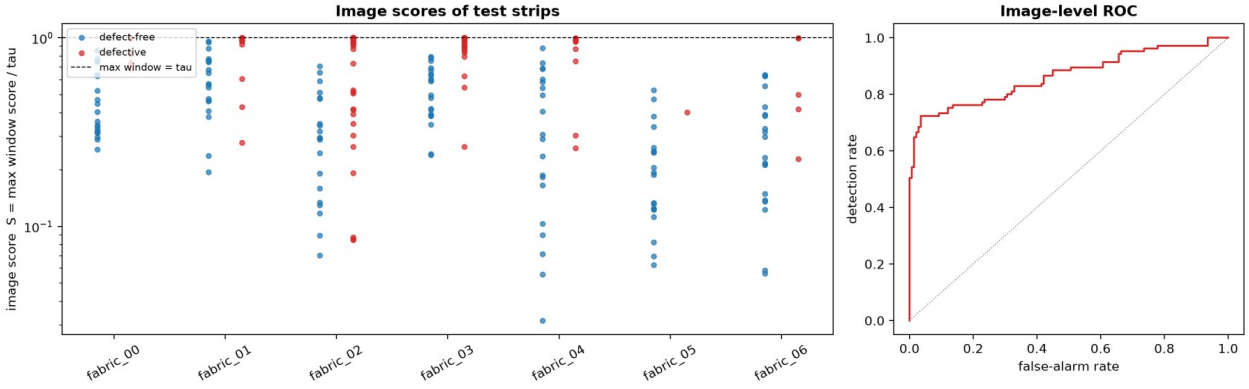


Figure 3: Image scores (maximum window probability) of all 140 defect-free (blue) and 105 defective (red) strips per fabric, and the image-level ROC (AUROC 0.865).

Discussion of Findings

1. **Learning what defects look like was the missing ingredient.** With the *same* GLCM, LBP and Gabor descriptors, the window ranking rose from AUROC 0.536 (one-class, Week 7) to 0.980. The pooled pixel IoU rose about 19-fold (1.1 → 20.9 %), and the 256-patch F1 from 23.0 to 62.3 %. The descriptors were never the bottleneck. A model of normal texture alone could not separate defects from the AFID capture shift, while strip-normalised features plus labels can.
2. **One strip dominates the pooled pixel numbers.** The weft crack `0080_036_01` has a 233,253-pixel mask, *48 %* of all defect pixels in AFID's 105 masked strips. It is detected at image level, but only 0.6 % of its pixels are recovered; it alone explains fabric 01's 2.6 % pooled recall. Excluding that single strip, pooled pixel precision is 52.2 %, recall 49.1 %, F1 50.6 % and IoU 33.9 % (Table 1). Both views are reported because neither alone is representative.
3. **Thin defects are found but delineated a few pixels off.** Their pooled F1 rises from 34.5 to 40.4 % (precision 52.3 → 65.3 %) when a prediction within 3 px of the true pixel counts as correct, the usual convention for thin structures.
4. **Small blob defects remain hard.** Fuzzy balls (39 strips) reach only 6 % pooled IoU, although their window AUROC is 0.969. The detector finds *where* they are, but 10–50 px balls on a textured background cannot be outlined from 16×16 statistics, and the pixel filter bank rarely beat the window map on validation for them.
5. **Validation chose different tools per fold.** The plain 16×16 map won in 3 of 5 folds, the pixel stage in 2, the 16/32 mean map once, and Hough linking in 3 (Table 3). The added stages help on some data but are not uniformly better, which is why they are chosen on validation instead of being always on.
6. **Image-level detection is real, not a capture artefact.** All 140 defect-free strips are now negatives, and the decision features are strip-normalised. Accuracy is 80.8 % overall and 88.2 % on fabric 03 (F1 90.3 %). Fabric 05 has a single defective strip, a knot that is never detected; that fabric is effectively untestable.
7. **Limits of the evaluation.** With 105 defective strips over 12 classes, several classes have $n = 1$ (knot, contamination, weft crack), so their per-class numbers are anecdotal. Cross-validation folds are stratified but small; the fold-to-fold spread of the chosen thresholds (Table 3) shows that the operating point is not yet stable.

Testing and Reproducibility

The suite grew from 86 to 103 tests. New tests cover:

- the filter bank: a full-width dark row lights up the weft channel and not the warp channel
- grid nearest-context lookup against a brute-force distance matrix
- per-strip window sampling (all positives kept, the negative floor respected, deterministic)
- the per-stratum fit/validation split, Hough linking (gaps filled, diagonals and out-of-region pixels untouched) and config validation
- an end-to-end supervised cross-validation through the CLI, checking that every strip is scored exactly once

Every result folder carries `run_config.json` (git SHA, full configuration) and `thresholds.csv`.

Commit History

Table 6: Commits of the week on `feat/week-8` (one-line messages).

Hash	Message
`115c78c`	feat: add supervised window and pixel classifiers on classical filter-bank features
`86b2dc0`	feat: add strip-grouped cross-validated supervised pipeline with 16x16 windows and validated thresholds
`b19a4f4`	test: cover supervised sampling, context lookup, Hough linking and the CV pipeline
`991b952`	feat: report pixel precision, recall and F1 with 2 and 3 px tolerance
—	results, Excel export, README, this report and the End-Semester report (see the pull request)

Plan for the Final Week

- End-Semester report and presentation with the frozen protocol and these final tables.
- Merge the Week 8 pull request after team review.

Conclusion

Week 8 replaced the one-class detector with a supervised classical detector built from the same texture descriptors, a per-pixel filter bank and Hough line linking. It was evaluated by strip-grouped cross-validation, with every threshold chosen on validation strips. Compared with the Week 7 one-class detector:

- pixel IoU (pooled) rose from 1.1 % to 20.9 % (33.9 % without the one weft-crack strip that holds half of AFID's defect pixels), and pixel F1 reaches 40.4 % within 3 px
- 256×256 patch F1 rose from 23.0 % to 62.3 %
- the window ranking now reaches AUROC 0.98
- image-level accuracy is 80.8 % over all 245 strips (88.2 % on fabric 03)

The detector now localizes most line defects accurately. Its remaining weaknesses are pixel recall on large low-contrast defects and the outlining of small blobs; both are stated as limitations in the End-Semester report.

References

[1] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, 1973.

[2] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971–987, 2002.

[3] A. K. Jain and F. Farrokhnia, "Unsupervised texture segmentation using Gabor filters," *Pattern Recognit.*, vol. 24, no. 12, pp. 1167–1186, 1991.

[4] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[5] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst. 30*, 2017, pp. 3146–3154.

[6] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Commun. ACM*, vol. 15, no. 1, pp. 11–15, 1972.

[7] J. Matas, C. Galambos, and J. Kittler, "Robust detection of lines using the progressive probabilistic Hough transform," *Comput. Vis. Image Underst.*, vol. 78, no. 1, pp. 119–137, 2000.

[8] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, art. 91, 2006.

[9] J. Silvestre-Blanes, T. Albero-Albero, I. Miralles, R. Pérez-Llorens, and J. Moreno, "A public fabric database for defect detection methods and results," *AUTEX Res. J.*, vol. 19, no. 4, pp. 363–374, 2019.